Curriculum

```c
/**
 * @file main.c
 * @brief Permutation Generator in C, architected to safety-critical standards.
 *
 * @author Dominic Alexander Cooper (Original Algorithm)
 * @author Gemini (Architectural Refactoring)
 *
 * @details
 * This program generates all possible character combinations for a given length,
 * based on a predefined character set. It is a complete rewrite of the original
 * concept to align with principles of safety-critical software design.
 */
#include <stdio.h>
#include <stdint.h> // For fixed-width integer types like uint64_t
#include <stdbool.h> // For bool type
#include <limits.h> // For UINT64_MAX
//============================================================================
// 1. CONFIGURATION AND DATA DEFINITIONS
//============================================================================
#define MAX_PERMUTATION_LENGTH 10
static const char ALPHABET[] = {
    'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
    'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
    'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',
    'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',
    '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', ' ', '\n'
};
static const uint64_t ALPHABET_SIZE = sizeof(ALPHABET) / sizeof(ALPHABET[0]);
//============================================================================
// 2. ABSTRACT INTERFACE FOR OUTPUT (OutputSink)
//============================================================================
typedef struct {
    void* context;
    bool (*write)(void* context, const char* buffer, uint64_t size);
    bool (*write_char)(void* context, char c);
} OutputSink;
//============================================================================
// 3. CORE LOGIC (Generator)
//============================================================================
static bool safe_uint64_power(uint64_t base, uint64_t exp, uint64_t* result) {
    *result = 1;
    for (uint64_t i = 0; i < exp; ++i) {
        if (*result > UINT64_MAX / base) {
            return false;
        }
        *result *= base;
    }
    return true;
```

```
}
static bool generate_permutations(uint64_t length, OutputSink* sink) {
    uint64_t num_combinations;
    if (!safe_uint64_power(ALPHABET_SIZE, length, &num_combinations)) {
        return false;
    }
    char current_perm[MAX_PERMUTATION_LENGTH];
    for (uint64_t i = 0; i < num_combinations; ++i) {
        uint64_t temp_row = i;
        for (int64_t j = length - 1; j >= 0; --j) {
            uint64_t char_index = temp_row % ALPHABET_SIZE;
            current_perm[j] = ALPHABET[char_index];
            temp_row /= ALPHABET_SIZE;
        }
        if (!sink->write(sink->context, current_perm, length)) {
            return false;
        }
        if (!sink->write_char(sink->context, '\n')) {
            return false;
        }
    }
    return true;
}
//============================================================================
// 4. CONCRETE IMPLEMENTATION OF OUTPUTSINK (FileSink)
//============================================================================
static bool file_sink_write(void* context, const char* buffer, uint64_t size) {
    FILE* p = (FILE*)context;
    return fwrite(buffer, sizeof(char), size, p) == size;
}
static bool file_sink_write_char(void* context, char c) {
    FILE* p = (FILE*)context;
    return fputc(c, p) != EOF;
}
static bool file_sink_init(OutputSink* sink, const char* filename) {
    FILE* p = fopen(filename, "w");
    if (p == NULL) {
        return false;
    }
    *sink = (OutputSink){
        .context = p,
        .write = file_sink_write,
        .write_char = file_sink_write_char
    };
    return true;
}
static void file_sink_close(OutputSink* sink) {
    if (sink && sink->context) {
        fclose((FILE*)sink->context);
        sink->context = NULL;
    }
}
//============================================================================
// 5. SYSTEM ASSEMBLER (main)
```

```c
//============================================================================
int main(void) {
    const uint64_t permutation_length = 4;
    if (permutation_length == 0 || permutation_length > MAX_PERMUTATION_LENGTH) {
        return 1;
    }
    OutputSink file_sink;
    if (!file_sink_init(&file_sink, "system_safe.txt")) {
        perror("Error opening file");
        return 1;
    }
    bool success = generate_permutations(permutation_length, &file_sink);
    file_sink_close(&file_sink);
    if (!success) {
        return 1;
    }
    return 0;
}
```

Curriculum (Enhanced)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>
#define x0 10
static const char x1[] = {
    '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'
};
static const uint64_t x2 = sizeof(x1) / sizeof(x1[0]);
typedef struct {
    void* x4;
    bool (*x5)(void* x4, const char* x6, uint64_t x7);
    bool (*x8)(void* x4, char x9);
} x3;
static bool x10(uint64_t x11, uint64_t x12, uint64_t* x13) {
    *x13 = 1;
    for (uint64_t x14 = 0; x14 < x12; ++x14) {
        if (*x13 > UINT64_MAX / x11) {
            return false;
        }
        *x13 *= x11;
    }
    return true;
}
static bool x15(uint64_t x16, x3* x17) {
    uint64_t x18;
    if (!x10(x2, x16, &x18)) {
        return false;
    }
    char x19[x0];
    for (uint64_t x14 = 0; x14 < x18; ++x14) {
        uint64_t x20 = x14;
        for (int64_t x21 = x16 - 1; x21 >= 0; --x21) {
            uint64_t x22 = x20 % x2;
            x19[x21] = x1[x22];
            x20 /= x2;
        }
        if (!x17->x5(x17->x4, x19, x16)) {
            return false;
        }
        if (!x17->x8(x17->x4, '\n')) {
            return false;
        }
    }
    return true;
}
static bool x23(void* x4, const char* x6, uint64_t x7) {
    FILE* x24 = (FILE*)x4;
```

```c
    return fwrite(x6, sizeof(char), x7, x24) == x7;
}
static bool x25(void* x4, char x9) {
    FILE* x24 = (FILE*)x4;
    return fputc(x9, x24) != EOF;
}
static bool x26(x3* x17, const char* x27) {
    FILE* x24 = fopen(x27, "w");
    if (x24 == NULL) {
        return false;
    }
    *x17 = (x3){
        .x4 = x24,
        .x5 = x23,
        .x8 = x25
    };
    return true;
}
static void x28(x3* x17) {
    if (x17 && x17->x4) {
        fclose((FILE*)x17->x4);
        x17->x4 = NULL;
    }
}
int main(void) {
    const uint64_t x30 = 7;
    if (x30 == 0 || x30 > x0) {
        return 1;
    }
    x3 x31;
    if (!x26(&x31, "x33.txt")) {
        perror("Error opening file");
        return 1;
    }
    bool x32 = x15(x30, &x31);
    x28(&x31);
    if (!x32) {
        return 1;
    }
    return 0;
}
```

Curriculum (Advanced)

```c
/*
 * configurator_v2.c
 *
 * A user-specifiable C generative intelligence fabric.
 *
 * Internal C identifiers intentionally preserve the xN object naming style used
 * by configurator.c. Runtime object names, input names, flow names, and output
 * names are user-specified through CLI flags, config files, or the micro UI.
 *
 * Build:
 *     cc -std=c11 -Wall -Wextra -pedantic -O2 configurator_v2.c -o
configurator_v2
 *
 * Examples:
 *     ./configurator_v2 --object-name x33 --input-name x1 --input-value 01 \
 *         --input-width 3 --flow-name x15 --flow-type cartesian \
 *         --output-name x31 --output-target x33.txt
 *
 *     ./configurator_v2 --sample-config x33.fabric
 *     ./configurator_v2 --config x33.fabric
 *     ./configurator_v2 --ui
 *
 * Config format:
 *     object_name=x33
 *     input_name=x1
 *     input_value=0123456789
 *     input_width=7
 *     flow_name=x15
 *     flow_type=cartesian
 *     output_name=x31
 *     output_target=x33.txt
 *     separator=\n
 *     end
 *
 * Supported flow_type values:
 *     cartesian  - generate every fixed-width combination from input_value
 *     literal    - write input_value once
 *     repeat     - write input_value input_width times
 *     reverse    - write input_value reversed once
 */

#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
```

```c
#define x0 64
#define x1 512
#define x2 32
#define x200 4096

typedef struct {
    void* x4;
    bool (*x5)(void* x4, const char* x6, uint64_t x7);
    bool (*x8)(void* x4, char x9);
    bool x10;
} x3;

typedef enum {
    x11 = 0,
    x12 = 1,
    x13 = 2,
    x14 = 3
} x15;

typedef struct {
    bool x16;
    char x17[x0];
    char x18[x0];
    char x19[x1];
    uint64_t x20;
    char x21[x0];
    x15 x22;
    char x23[x0];
    char x24[x1];
    char x25[x0];
    char x26[x1];
    char x27[x1];
} x28;

typedef struct {
    x28 x29[x2];
    uint64_t x30;
} x31;

static bool x32(char* x33, uint64_t x34, const char* x35) {
    size_t x36;

    if (x33 == NULL || x35 == NULL || x34 == 0) {
        return false;
    }

    x36 = strlen(x35);
    if (x36 >= x34) {
        return false;
    }

    memcpy(x33, x35, x36 + 1);
    return true;
```

```
}

static int x37(int x38) {
    return tolower((unsigned char)x38);
}

static bool x39(const char* x40, const char* x41) {
    if (x40 == NULL || x41 == NULL) {
        return false;
    }

    while (*x40 && *x41) {
        if (x37(*x40) != x37(*x41)) {
            return false;
        }
        ++x40;
        ++x41;
    }

    return *x40 == '\0' && *x41 == '\0';
}

static char* x42(char* x43) {
    char* x44;
    size_t x45;

    if (x43 == NULL) {
        return NULL;
    }

    while (*x43 && isspace((unsigned char)*x43)) {
        ++x43;
    }

    x45 = strlen(x43);
    while (x45 > 0 && isspace((unsigned char)x43[x45 - 1])) {
        x43[x45 - 1] = '\0';
        --x45;
    }

    x44 = strchr(x43, '#');
    if (x44 != NULL) {
        *x44 = '\0';
        x45 = strlen(x43);
        while (x45 > 0 && isspace((unsigned char)x43[x45 - 1])) {
            x43[x45 - 1] = '\0';
            --x45;
        }
    }

    return x43;
}

static bool x46(char* x47, char** x48, char** x49) {
```

```
    char* x50;

    if (x47 == NULL || x48 == NULL || x49 == NULL) {
        return false;
    }

    x50 = strchr(x47, '=');
    if (x50 != NULL) {
        *x50 = '\0';
        *x48 = x42(x47);
        *x49 = x42(x50 + 1);
        return **x48 != '\0';
    }

    x50 = x47;
    while (*x50 && !isspace((unsigned char)*x50)) {
        ++x50;
    }

    if (*x50 == '\0') {
        *x48 = x42(x47);
        *x49 = NULL;
        return **x48 != '\0';
    }

    *x50 = '\0';
    ++x50;
    *x48 = x42(x47);
    *x49 = x42(x50);

    return **x48 != '\0';
}

static bool x51(const char* x52, uint64_t* x53) {
    char* x54;
    unsigned long long x55;

    if (x52 == NULL || x53 == NULL || *x52 == '\0') {
        return false;
    }

    while (*x52 && isspace((unsigned char)*x52)) {
        ++x52;
    }

    if (*x52 == '-') {
        return false;
    }

    x55 = strtoull(x52, &x54, 10);
    if (x54 == x52 || *x42(x54) != '\0') {
        return false;
    }
```

```c
    if (x55 > UINT64_MAX) {
        return false;
    }

    *x53 = (uint64_t)x55;
    return true;
}

static bool x56(char* x57, uint64_t x58, const char* x59) {
    uint64_t x60 = 0;

    if (x57 == NULL || x59 == NULL || x58 == 0) {
        return false;
    }

    while (*x59) {
        char x61 = *x59++;

        if (x61 == '\\') {
            x61 = *x59++;
            if (x61 == '\0') {
                x61 = '\\';
                --x59;
            } else if (x61 == 'n') {
                x61 = '\n';
            } else if (x61 == 't') {
                x61 = '\t';
            } else if (x61 == 'r') {
                x61 = '\r';
            } else if (x61 == '\\') {
                x61 = '\\';
            }
        }

        if (x60 + 1 >= x58) {
            return false;
        }

        x57[x60++] = x61;
    }

    x57[x60] = '\0';
    return true;
}

static void x62(x28* x63) {
    if (x63 == NULL) {
        return;
    }

    memset(x63, 0, sizeof(*x63));
    x63->x16 = true;
    (void)x32(x63->x17, sizeof(x63->x17), "x33");
    (void)x32(x63->x18, sizeof(x63->x18), "x1");
```

```c
    (void)x32(x63->x19, sizeof(x63->x19), "0123456789");
    x63->x20 = 7;
    (void)x32(x63->x21, sizeof(x63->x21), "x15");
    x63->x22 = x11;
    (void)x32(x63->x23, sizeof(x63->x23), "x31");
    (void)x32(x63->x24, sizeof(x63->x24), "x33.txt");
    (void)x32(x63->x25, sizeof(x63->x25), "\n");
    (void)x32(x63->x26, sizeof(x63->x26), "");
    (void)x32(x63->x27, sizeof(x63->x27), "");
}

static void x64(x31* x65) {
    if (x65 != NULL) {
        memset(x65, 0, sizeof(*x65));
    }
}

static bool x66(x31* x67, const x28* x68) {
    if (x67 == NULL || x68 == NULL || x67->x30 >= x2) {
        return false;
    }

    x67->x29[x67->x30++] = *x68;
    return true;
}

static bool x69(const char* x70, x15* x71) {
    if (x70 == NULL || x71 == NULL) {
        return false;
    }

    if (x39(x70, "cartesian") || x39(x70, "product") || x39(x70, "combinations"))
{
        *x71 = x11;
        return true;
    }

    if (x39(x70, "literal") || x39(x70, "echo")) {
        *x71 = x12;
        return true;
    }

    if (x39(x70, "repeat")) {
        *x71 = x13;
        return true;
    }

    if (x39(x70, "reverse")) {
        *x71 = x14;
        return true;
    }

    return false;
}
```

```c
static bool x72(void* x4, const char* x6, uint64_t x7) {
    FILE* x73 = (FILE*)x4;
    return fwrite(x6, sizeof(char), (size_t)x7, x73) == x7;
}

static bool x74(void* x4, char x9) {
    FILE* x73 = (FILE*)x4;
    return fputc((unsigned char)x9, x73) != EOF;
}

static bool x75(x3* x76, const char* x77) {
    FILE* x73;

    if (x76 == NULL || x77 == NULL || *x77 == '\0') {
        return false;
    }

    if (x39(x77, "stdout") || x39(x77, "-")) {
        *x76 = (x3){
            .x4 = stdout,
            .x5 = x72,
            .x8 = x74,
            .x10 = false
        };
        return true;
    }

    x73 = fopen(x77, "w");
    if (x73 == NULL) {
        return false;
    }

    *x76 = (x3){
        .x4 = x73,
        .x5 = x72,
        .x8 = x74,
        .x10 = true
    };

    return true;
}

static void x78(x3* x79) {
    if (x79 != NULL && x79->x4 != NULL && x79->x10) {
        fclose((FILE*)x79->x4);
        x79->x4 = NULL;
        x79->x10 = false;
    }
}

static bool x80(x3* x81, const char* x82) {
    uint64_t x83;
```

```c
    if (x81 == NULL || x82 == NULL) {
        return false;
    }

    x83 = (uint64_t)strlen(x82);
    if (x83 == 0) {
        return true;
    }

    return x81->x5(x81->x4, x82, x83);
}

static bool x84(uint64_t x85, uint64_t x86, uint64_t* x87) {
    uint64_t x88;

    if (x87 == NULL || x85 == 0) {
        return false;
    }

    *x87 = 1;
    for (x88 = 0; x88 < x86; ++x88) {
        if (*x87 > UINT64_MAX / x85) {
            return false;
        }
        *x87 *= x85;
    }

    return true;
}

static bool x89(const x28* x90) {
    uint64_t x91;

    if (x90 == NULL) {
        return false;
    }

    if (x90->x17[0] == '\0' || x90->x18[0] == '\0' ||
        x90->x21[0] == '\0' || x90->x23[0] == '\0' ||
        x90->x24[0] == '\0') {
        return false;
    }

    if (x90->x20 == 0) {
        return false;
    }

    if (x90->x22 == x11) {
        if (x90->x20 > x0) {
            return false;
        }
        if (strlen(x90->x19) == 0) {
            return false;
        }
```

```c
        if (!x84((uint64_t)strlen(x90->x19), x90->x20, &x91)) {
            return false;
        }
    }

    return true;
}

static bool x92(x3* x93, const x28* x94, const char* x95, uint64_t x96) {
    if (x93 == NULL || x94 == NULL || x95 == NULL) {
        return false;
    }

    if (!x80(x93, x94->x26)) {
        return false;
    }

    if (x96 > 0 && !x93->x5(x93->x4, x95, x96)) {
        return false;
    }

    if (!x80(x93, x94->x27)) {
        return false;
    }

    if (!x80(x93, x94->x25)) {
        return false;
    }

    return true;
}

static bool x97(const x28* x98, x3* x99) {
    uint64_t x100;
    uint64_t x101;
    uint64_t x102;
    char x103[x0 + 1];

    if (x98 == NULL || x99 == NULL) {
        return false;
    }

    x100 = (uint64_t)strlen(x98->x19);
    if (!x84(x100, x98->x20, &x101)) {
        return false;
    }

    for (x102 = 0; x102 < x101; ++x102) {
        uint64_t x104 = x102;
        int64_t x105;

        for (x105 = (int64_t)x98->x20 - 1; x105 >= 0; --x105) {
            uint64_t x106 = x104 % x100;
            x103[x105] = x98->x19[x106];
```

```
            x104 /= x100;
        }

        x103[x98->x20] = '\0';

        if (!x92(x99, x98, x103, x98->x20)) {
            return false;
        }
    }

    return true;
}

static bool x107(const x28* x108, x3* x109) {
    if (x108 == NULL || x109 == NULL) {
        return false;
    }

    return x92(x109, x108, x108->x19, (uint64_t)strlen(x108->x19));
}

static bool x110(const x28* x111, x3* x112) {
    uint64_t x113;

    if (x111 == NULL || x112 == NULL) {
        return false;
    }

    for (x113 = 0; x113 < x111->x20; ++x113) {
        if (!x92(x112, x111, x111->x19, (uint64_t)strlen(x111->x19))) {
            return false;
        }
    }

    return true;
}

static bool x114(const x28* x115, x3* x116) {
    size_t x117;
    char x118[x1];
    size_t x119;

    if (x115 == NULL || x116 == NULL) {
        return false;
    }

    x117 = strlen(x115->x19);
    if (x117 >= sizeof(x118)) {
        return false;
    }

    for (x119 = 0; x119 < x117; ++x119) {
        x118[x119] = x115->x19[x117 - 1 - x119];
    }
```

```
    x118[x117] = '\0';

    return x92(x116, x115, x118, (uint64_t)x117);
}

static bool x120(const x28* x121) {
    x3 x122;
    bool x123;

    if (!x89(x121)) {
        fprintf(stderr, "Invalid object specification: %s\n", x121 ? x121->x17 : "
(null)");
        return false;
    }

    if (!x75(&x122, x121->x24)) {
        perror("Error opening output target");
        return false;
    }

    if (x121->x22 == x11) {
        x123 = x97(x121, &x122);
    } else if (x121->x22 == x12) {
        x123 = x107(x121, &x122);
    } else if (x121->x22 == x13) {
        x123 = x110(x121, &x122);
    } else if (x121->x22 == x14) {
        x123 = x114(x121, &x122);
    } else {
        x123 = false;
    }

    x78(&x122);
    return x123;
}

static bool x124(const x31* x125) {
    uint64_t x126;

    if (x125 == NULL || x125->x30 == 0) {
        return false;
    }

    for (x126 = 0; x126 < x125->x30; ++x126) {
        if (!x120(&x125->x29[x126])) {
            return false;
        }
    }

    return true;
}

static bool x127(x28* x128, const char* x129, const char* x130) {
    uint64_t x131;
```

```
    x15 x132;

    if (x128 == NULL || x129 == NULL || x130 == NULL) {
        return false;
    }

    if (x39(x129, "object") || x39(x129, "object_name") || x39(x129, "name")) {
        return x32(x128->x17, sizeof(x128->x17), x130);
    }

    if (x39(x129, "input_name")) {
        return x32(x128->x18, sizeof(x128->x18), x130);
    }

    if (x39(x129, "input") || x39(x129, "input_value") || x39(x129, "alphabet")) {
        return x56(x128->x19, sizeof(x128->x19), x130);
    }

    if (x39(x129, "input_width") || x39(x129, "width") ||
        x39(x129, "length") || x39(x129, "depth")) {
        if (!x51(x130, &x131)) {
            return false;
        }
        x128->x20 = x131;
        return true;
    }

    if (x39(x129, "flow_name")) {
        return x32(x128->x21, sizeof(x128->x21), x130);
    }

    if (x39(x129, "flow") || x39(x129, "flow_type") || x39(x129,
"processing_flow")) {
        if (!x69(x130, &x132)) {
            return false;
        }
        x128->x22 = x132;
        return true;
    }

    if (x39(x129, "output_name")) {
        return x32(x128->x23, sizeof(x128->x23), x130);
    }

    if (x39(x129, "output") || x39(x129, "output_target") || x39(x129, "file")) {
        return x32(x128->x24, sizeof(x128->x24), x130);
    }

    if (x39(x129, "separator") || x39(x129, "sep")) {
        return x56(x128->x25, sizeof(x128->x25), x130);
    }

    if (x39(x129, "prefix")) {
        return x56(x128->x26, sizeof(x128->x26), x130);
```

```
    }

    if (x39(x129, "suffix")) {
        return x56(x128->x27, sizeof(x128->x27), x130);
    }

    return false;
}

static bool x133(const char* x134, x31* x135) {
    FILE* x136;
    char x137[x200];
    uint64_t x138 = 0;
    x28 x139;
    bool x140 = false;

    if (x134 == NULL || x135 == NULL) {
        return false;
    }

    x136 = fopen(x134, "r");
    if (x136 == NULL) {
        return false;
    }

    x64(x135);
    x62(&x139);

    while (fgets(x137, sizeof(x137), x136) != NULL) {
        char* x141;
        char* x142;
        char* x143;

        ++x138;
        x141 = x42(x137);

        if (*x141 == '\0') {
            continue;
        }

        if (x39(x141, "end")) {
            if (!x66(x135, &x139)) {
                fclose(x136);
                return false;
            }
            x62(&x139);
            x140 = false;
            continue;
        }

        if (!x46(x141, &x142, &x143) || x143 == NULL) {
            fprintf(stderr, "Invalid config line %llu\n", (unsigned long
long)x138);
            fclose(x136);
```

```
            return false;
        }

        if (x39(x142, "object") || x39(x142, "object_name")) {
            if (x140) {
                if (!x66(x135, &x139)) {
                    fclose(x136);
                    return false;
                }
                x62(&x139);
            }
            x140 = true;
        } else {
            x140 = true;
        }

        if (!x127(&x139, x142, x143)) {
            fprintf(stderr, "Invalid config key or value on line %llu: %s\n",
                    (unsigned long long)x138, x142);
            fclose(x136);
            return false;
        }
    }

    if (x140) {
        if (!x66(x135, &x139)) {
            fclose(x136);
            return false;
        }
    }

    fclose(x136);
    return x135->x30 > 0;
}

static bool x144(const char* x145, char* x146, uint64_t x147, const char* x148) {
    char x149[x200];
    size_t x150;

    if (x145 == NULL || x146 == NULL || x147 == 0 || x148 == NULL) {
        return false;
    }

    printf("%s [%s]: ", x145, x148);
    fflush(stdout);

    if (fgets(x149, sizeof(x149), stdin) == NULL) {
        return false;
    }

    x150 = strlen(x149);
    if (x150 > 0 && x149[x150 - 1] == '\n') {
        x149[x150 - 1] = '\0';
    }
```

```
    if (x149[0] == '\0') {
        return x32(x146, x147, x148);
    }

    return x32(x146, x147, x149);
}

static bool x151(const char* x152, uint64_t* x153, uint64_t x154) {
    char x155[x0];
    char x156[x0];

    if (x152 == NULL || x153 == NULL) {
        return false;
    }

    snprintf(x156, sizeof(x156), "%llu", (unsigned long long)x154);

    if (!x144(x152, x155, sizeof(x155), x156)) {
        return false;
    }

    return x51(x155, x153);
}

static bool x157(x31* x158) {
    x28 x159;
    char x160[x0];
    bool x161 = true;

    if (x158 == NULL) {
        return false;
    }

    x64(x158);

    while (x161 && x158->x30 < x2) {
        x62(&x159);

        if (!x144("object_name", x159.x17, sizeof(x159.x17), x159.x17)) {
            return false;
        }
        if (!x144("input_name", x159.x18, sizeof(x159.x18), x159.x18)) {
            return false;
        }
        if (!x144("input_value", x159.x19, sizeof(x159.x19), x159.x19)) {
            return false;
        }
        if (!x151("input_width", &x159.x20, x159.x20)) {
            return false;
        }
        if (!x144("flow_name", x159.x21, sizeof(x159.x21), x159.x21)) {
            return false;
        }
```

```
        if (!x144("flow_type cartesian|literal|repeat|reverse", x160,
sizeof(x160), "cartesian")) {
            return false;
        }
        if (!x69(x160, &x159.x22)) {
            fprintf(stderr, "Unknown flow_type: %s\n", x160);
            return false;
        }
        if (!x144("output_name", x159.x23, sizeof(x159.x23), x159.x23)) {
            return false;
        }
        if (!x144("output_target file|stdout|-", x159.x24, sizeof(x159.x24),
x159.x24)) {
            return false;
        }
        if (!x144("separator", x160, sizeof(x160), "\\n")) {
            return false;
        }
        if (!x56(x159.x25, sizeof(x159.x25), x160)) {
            return false;
        }
        if (!x144("prefix", x160, sizeof(x160), "")) {
            return false;
        }
        if (!x56(x159.x26, sizeof(x159.x26), x160)) {
            return false;
        }
        if (!x144("suffix", x160, sizeof(x160), "")) {
            return false;
        }
        if (!x56(x159.x27, sizeof(x159.x27), x160)) {
            return false;
        }

        if (!x66(x158, &x159)) {
            return false;
        }

        if (!x144("add another object? y|n", x160, sizeof(x160), "n")) {
            return false;
        }

        x161 = x39(x160, "y") || x39(x160, "yes");
    }

    return x158->x30 > 0;
}

static bool x162(const char* x163) {
    FILE* x164;

    if (x163 == NULL) {
        return false;
    }
```

```
    x164 = fopen(x163, "w");
    if (x164 == NULL) {
        return false;
    }

    fprintf(x164,
            "# configurator_v2 fabric config\n"
            "object_name=x33\n"
            "input_name=x1\n"
            "input_value=0123456789\n"
            "input_width=3\n"
            "flow_name=x15\n"
            "flow_type=cartesian\n"
            "output_name=x31\n"
            "output_target=x33.txt\n"
            "separator=\\n\n"
            "prefix=\n"
            "suffix=\n"
            "end\n"
            "\n"
            "object_name=x34\n"
            "input_name=x35\n"
            "input_value=emerge\n"
            "input_width=1\n"
            "flow_name=x36\n"
            "flow_type=reverse\n"
            "output_name=x37\n"
            "output_target=stdout\n"
            "separator=\\n\n"
            "end\n");

    fclose(x164);
    return true;
}

static void x165(const char* x166) {
    fprintf(stderr,
            "Usage:\n"
            "  %s --ui\n"
            "  %s --config <path>\n"
            "  %s --sample-config <path>\n"
            "  %s [object flags]\n"
            "\n"
            "Object flags:\n"
            "  --object-name <name>       runtime object name\n"
            "  --input-name <name>        runtime input name\n"
            "  --input-value <value>      object input payload/alphabet\n"
            "  --input-width <n>          width/count/depth\n"
            "  --flow-name <name>         runtime processing-flow name\n"
            "  --flow-type <type>         cartesian|literal|repeat|reverse\n"
            "  --output-name <name>       runtime output name\n"
            "  --output-target <target>   file path, stdout, or -\n"
            "  --separator <text>         supports \\\\n, \\\\t, \\\\r,
```

```
\\\\\\\\\n"
            "  --prefix <text>            supports escapes\n"
            "  --suffix <text>            supports escapes\n"
            "\n"
            "Aliases:\n"
            "  -n <name>  -i <value>  -w <n>  -f <type>  -o <target>\n",
            x166, x166, x166, x166);
}

static int x167(int x168, char** x169, x31* x170, bool* x171) {
    int x172;
    x28 x173;
    bool x174 = false;

    if (x170 == NULL || x171 == NULL) {
        return 1;
    }

    *x171 = false;
    x64(x170);

    if (x168 <= 1) {
        x165(x169[0]);
        return 0;
    }

    for (x172 = 1; x172 < x168; ++x172) {
        if (x39(x169[x172], "--help") || x39(x169[x172], "-h")) {
            x165(x169[0]);
            return 0;
        }

        if (x39(x169[x172], "--ui")) {
            if (!x157(x170)) {
                return 1;
            }
            *x171 = true;
            return 0;
        }

        if (x39(x169[x172], "--config")) {
            if (x172 + 1 >= x168) {
                x165(x169[0]);
                return 1;
            }
            if (!x133(x169[x172 + 1], x170)) {
                perror("Error loading config");
                return 1;
            }
            *x171 = true;
            return 0;
        }

        if (x39(x169[x172], "--sample-config")) {
```

```
            if (x172 + 1 >= x168) {
                x165(x169[0]);
                return 1;
            }
            if (!x162(x169[x172 + 1])) {
                perror("Error writing sample config");
                return 1;
            }
            *x171 = false;
            return 0;
        }
    }

    x62(&x173);

    for (x172 = 1; x172 < x168; ++x172) {
        const char* x175 = x169[x172];
        const char* x176 = NULL;

        if (x172 + 1 < x168) {
            x176 = x169[x172 + 1];
        }

        if (x176 == NULL) {
            x165(x169[0]);
            return 1;
        }

        if (x39(x175, "--object-name") || x39(x175, "-n")) {
            if (!x127(&x173, "object_name", x176)) {
                return 1;
            }
        } else if (x39(x175, "--input-name")) {
            if (!x127(&x173, "input_name", x176)) {
                return 1;
            }
        } else if (x39(x175, "--input-value") || x39(x175, "-i")) {
            if (!x127(&x173, "input_value", x176)) {
                return 1;
            }
        } else if (x39(x175, "--input-width") || x39(x175, "-w")) {
            if (!x127(&x173, "input_width", x176)) {
                return 1;
            }
        } else if (x39(x175, "--flow-name")) {
            if (!x127(&x173, "flow_name", x176)) {
                return 1;
            }
        } else if (x39(x175, "--flow-type") || x39(x175, "-f")) {
            if (!x127(&x173, "flow_type", x176)) {
                return 1;
            }
        } else if (x39(x175, "--output-name")) {
            if (!x127(&x173, "output_name", x176)) {
```

```
                return 1;
            }
        } else if (x39(x175, "--output-target") || x39(x175, "-o")) {
            if (!x127(&x173, "output_target", x176)) {
                return 1;
            }
        } else if (x39(x175, "--separator")) {
            if (!x127(&x173, "separator", x176)) {
                return 1;
            }
        } else if (x39(x175, "--prefix")) {
            if (!x127(&x173, "prefix", x176)) {
                return 1;
            }
        } else if (x39(x175, "--suffix")) {
            if (!x127(&x173, "suffix", x176)) {
                return 1;
            }
        } else {
            fprintf(stderr, "Unknown flag: %s\n", x175);
            x165(x169[0]);
            return 1;
        }

        ++x172;
        x174 = true;
    }

    if (!x174) {
        x165(x169[0]);
        return 0;
    }

    if (!x66(x170, &x173)) {
        return 1;
    }

    *x171 = true;
    return 0;
}

int main(int x177, char** x178) {
    x31 x179;
    bool x180;
    int x181;

    x181 = x167(x177, x178, &x179, &x180);
    if (x181 != 0) {
        return x181;
    }

    if (!x180) {
        return 0;
    }
```

```
    if (!x124(&x179)) {
        return 1;
    }

    return 0;
}
```

C Source Code Reading Exercise — Permutation Generator

Focus: code comprehension, overflow safety, output abstraction, file I/O correctness, and testing
Source curriculum: Permutation Generator in C, architected to safety-critical standards.
Learning Engine basis: Mastery-loop exercise: definitions → focused reading → MCQ with rationales → open questions → testing/homework.
Estimated time: 45–60 minutes plus optional coding extension.

Learning outcomes

By the end of this exercise, you should be able to:

- Explain what the program generates and how base-N counting maps integers to character sequences.
- Trace the flow from `main` through `generate_permutations` to the output sink.
- Justify the overflow guard in `safe_uint64_power` and identify edge cases.
- Describe why `OutputSink` improves testability and separation of concerns.
- Review file-writing code for correctness, failure handling, and resource cleanup.

How to use this exercise

Read the source excerpt in the appendix first. Then complete Sections A–E without compiling. Use Section F only after you have written your own answers.

Section A — Guided code survey

A1. In one sentence, state the program's observable output when `permutation_length` is `4`.

> Answer:

A2. Identify the five major sections of the program and write the responsibility of each section.

Section	Responsibility
1. Configuration and data definitions	
2. Abstract interface for output (`OutputSink`)	
3. Core logic / generator	
4. Concrete `FileSink` implementation	
5. System assembler (`main`)	

A3. What is the value of `ALPHABET_SIZE`? Show how you counted it.

> Answer:

A4. Why does the alphabet include both a space character and a newline character? What consequence does this have for the output file?

> Answer:

A5. What invariant should hold for `current_perm` immediately before each call to `sink->write`?

> Answer:

Section B — Trace the generator

For this section, pretend the alphabet is shortened to `{'a','b','c'}` and `length = 2`. This makes tracing manageable while preserving the algorithm.

`i`	`temp_row` evolution	`j = 1` char	`j = 0` char	generated string
0				
1				
2				
3				
4				
5				
6				
7				
8				

B1. Explain why the inner loop counts backward from `length - 1` to `0`.

> Answer:

B2. Describe the relationship between `i`, repeated division by `ALPHABET_SIZE`, and base-N representation.

> Answer:

B3. What would change if the inner loop counted upward from `0` to `length - 1`?

> Answer:

Section C — Multiple choice with justification

Choose one option for each question. Then write one sentence justifying why your option is correct and one sentence explaining the misconception behind one incorrect option.

C1

Why does `safe_uint64_power` test `*result > UINT64_MAX / base` before multiplication?

A. To avoid division by zero in the next loop iteration.
B. To detect whether multiplying by `base` would overflow `uint64_t`.
C. To ensure `exp` is less than `base`.
D. To round the result down to a safe value.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C2

What design role does `OutputSink` play?

A. It hides the alphabet from the generator.
B. It lets the generator write to any compatible destination without knowing whether it is a file, buffer, or test double.
C. It automatically retries failed writes.
D. It prevents all memory errors at compile time.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C3

What does `fwrite(buffer, sizeof(char), size, p) == size` check?

A. That exactly `size` characters were written.
B. That the file contains no newline characters.
C. That the output file was opened in binary mode.
D. That `buffer` is null-terminated.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C4

Which input validation is performed in `main` before generation?

A. It checks that the filename is non-empty.
B. It checks that `permutation_length` is nonzero and not greater than `MAX_PERMUTATION_LENGTH`.
C. It checks that the alphabet contains no duplicate characters.
D. It checks that disk space is sufficient for the output.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C5

Which issue is most worth discussing in a safety-critical review of this code?

A. `current_perm` is not null-terminated, but the program writes by explicit length.
B. The generator relies on a huge output size, so runtime and disk exhaustion are possible even when integer overflow is prevented.
C. `uint64_t` cannot represent positive numbers.
D. `fputc` always fails after `fwrite`.

> Selected option:
> Justification:
> Misconception in one incorrect option:

Section D — Open response / construction tasks

D1. Overflow proof sketch. Write a short proof that the guard in `safe_uint64_power` prevents wraparound when `base > 0`. Include the precondition you need.

> Answer:

D2. Failure path analysis. List every place generation can return `false`. For each place, state what resource is open at that moment and who is responsible for cleanup.

> Answer:

D3. Test double design. Design an in-memory `OutputSink` that stores output into a buffer. Specify its context struct fields and two behaviours you would assert in tests.

> Answer:

D4. Property-based test. Propose a property-based test that would catch a broken overflow guard. Include generator inputs, expected invariant, and at least two edge cases.

> Answer:

D5. Safety-critical improvement. Recommend two changes that would make the program safer for production use. Explain the risk each change addresses.

> Answer:

Section E — Code review checklist

Item	Question	Evidence from code	Status
Bounds	Can all array writes to `current_perm` be shown within `0..MAX_PERMUTATION_LENGTH-1`?		
Overflow	Is every arithmetic growth step guarded before it can overflow?		
I/O	Are partial writes and write failures detected?		

Item	Question	Evidence from code	Status
Cleanup	Is the file closed on both success and generation failure?		
Testability	Can generator logic be tested without writing a real file?		
Scalability	Can the program avoid infeasible output volumes?		

Section F — Answer key and marking guide

Use this section after attempting the exercise. Award partial credit for precise reasoning even where wording differs.

MCQ key

Question	Answer	Core rationale
C1	B	The comparison checks whether `result × base` would exceed `UINT64_MAX` before multiplication happens.
C2	B	The sink interface decouples generation from the output destination and enables test doubles.
C3	A	`fwrite` returns the number of elements written; here each element is one `char`.
C4	B	`main` rejects zero length and values above `MAX_PERMUTATION_LENGTH`.
C5	B	Overflow safety does not guarantee feasible runtime, output size, or disk capacity.

Open-response rubric

Task	4 points	2 points	0–1 point
D1	States precondition `base > 0` and explains division guard before multiplication.	Mentions overflow guard but proof is incomplete.	Only restates code or misses overflow mechanism.
D2	Finds overflow, write, and `write_char` failures; notes `main` closes file after generation.	Finds some failure paths or cleanup partly.	Misses most error paths.
D3	Defines context buffer/capacity/length and useful assertions.	Gives a plausible sink with limited assertions.	Does not preserve interface contract.
D4	Uses edge cases near `UINT64_MAX` and checks false/no wraparound.	Has tests but weak edge cases.	Only tests ordinary small values.
D5	Links changes to risks such as disk exhaustion, null sink, base zero, filename/config validation.	Suggests changes with vague risk explanation.	Cosmetic-only changes.

Spaced retrieval prompts

- Tomorrow: Explain the overflow guard without looking at the code.
- In three days: Re-trace the shortened alphabet example for `length = 3`.
- In one week: Write a memory-backed `OutputSink` from scratch.
- In two weeks: Review the program for infeasible output size and propose a mitigation.

Appendix — Source excerpt for reading

Read this source carefully before completing the exercise. Line numbers are not included, so refer to function names and code fragments in your answers.

```c
/**
 * @file main.c
 * @brief Permutation Generator in C, architected to safety-critical standards.
 *
 * @author Dominic Alexander Cooper (Original Algorithm)
 * @author Gemini (Architectural Refactoring)
 *
 * @details
 * This program generates all possible character combinations for a given length,
 * based on a predefined character set. It is a complete rewrite of the original
 * concept to align with principles of safety-critical software design.
 */
#include <stdio.h>
#include <stdint.h>  // For fixed-width integer types like uint64_t
#include <stdbool.h> // For bool type
#include <limits.h>  // For UINT64_MAX

//============================================================================
// 1. CONFIGURATION AND DATA DEFINITIONS
//============================================================================
#define MAX_PERMUTATION_LENGTH 10

static const char ALPHABET[] = {
    'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
    'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
    'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',
    'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',
    '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', ' ', '\n'
};

static const uint64_t ALPHABET_SIZE = sizeof(ALPHABET) / sizeof(ALPHABET[0]);

//============================================================================
// 2. ABSTRACT INTERFACE FOR OUTPUT (OutputSink)
//============================================================================
typedef struct {
    void* context;
    bool (*write)(void* context, const char* buffer, uint64_t size);
    bool (*write_char)(void* context, char c);
} OutputSink;
```

```cpp
//============================================================================
// 3. CORE LOGIC (Generator)
//============================================================================
static bool safe_uint64_power(uint64_t base, uint64_t exp, uint64_t* result) {
    *result = 1;

    for (uint64_t i = 0; i < exp; ++i) {
        if (*result > UINT64_MAX / base) {
            return false;
        }

        *result *= base;
    }

    return true;
}

static bool generate_permutations(uint64_t length, OutputSink* sink) {
    uint64_t num_combinations;

    if (!safe_uint64_power(ALPHABET_SIZE, length, &num_combinations)) {
        return false;
    }

    char current_perm[MAX_PERMUTATION_LENGTH];

    for (uint64_t i = 0; i < num_combinations; ++i) {
        uint64_t temp_row = i;

        for (int64_t j = length - 1; j >= 0; --j) {
            uint64_t char_index = temp_row % ALPHABET_SIZE;
            current_perm[j] = ALPHABET[char_index];
            temp_row /= ALPHABET_SIZE;
        }

        if (!sink->write(sink->context, current_perm, length)) {
            return false;
        }

        if (!sink->write_char(sink->context, '\n')) {
            return false;
        }
    }

    return true;
}

//============================================================================
// 4. CONCRETE IMPLEMENTATION OF OUTPUTSINK (FileSink)
//============================================================================
static bool file_sink_write(void* context, const char* buffer, uint64_t size) {
    FILE* p = (FILE*)context;
    return fwrite(buffer, sizeof(char), size, p) == size;
}
```

```c
static bool file_sink_write_char(void* context, char c) {
    FILE* p = (FILE*)context;
    return fputc(c, p) != EOF;
}

static bool file_sink_init(OutputSink* sink, const char* filename) {
    FILE* p = fopen(filename, "w");

    if (p == NULL) {
        return false;
    }

    *sink = (OutputSink){
        .context = p,
        .write = file_sink_write,
        .write_char = file_sink_write_char
    };

    return true;
}

static void file_sink_close(OutputSink* sink) {
    if (sink && sink->context) {
        fclose((FILE*)sink->context);
        sink->context = NULL;
    }
}

//============================================================================
// 5. SYSTEM ASSEMBLER (main)
//============================================================================
int main(void) {
    const uint64_t permutation_length = 4;

    if (permutation_length == 0 || permutation_length > MAX_PERMUTATION_LENGTH) {
        return 1;
    }

    OutputSink file_sink;

    if (!file_sink_init(&file_sink, "system_safe.txt")) {
        perror("Error opening file");
        return 1;
    }

    bool success = generate_permutations(permutation_length, &file_sink);

    file_sink_close(&file_sink);

    if (!success) {
        return 1;
    }
```

```
    return 0;
}
```

Advanced C Curriculum Exercise

Source-code comprehension for purpose, architecture, and API-oriented use cases

Student: _____
Date: _____

Source under study: `curriculum_advanced.pdf`, which contains `configurator_v2.c`: a configurable C program that can generate outputs through command-line flags, config files, or a micro UI. The source supports flow types including `cartesian`, `literal`, `repeat`, and `reverse`, and abstracts output through a sink-like interface.

Estimated time: 60-90 minutes.
Prerequisites: C structs, enums, function pointers, file I/O, CLI parsing, bounded string handling, and integer overflow guards.

Learning objectives

- Explain the high-level purpose of `configurator_v2.c` without being distracted by `xN`-style internal identifiers.
- Trace how user-facing API inputs - CLI flags, config-file keys, and UI prompts - become an executable object specification.
- Identify core API-oriented design choices: stable command surface, config schema, output abstraction, validation boundary, and dispatch by flow type.
- Evaluate plausible use cases for generated outputs in API-oriented software engineering.
- Propose tests and design improvements that preserve the program's external contract while improving safety and maintainability.

Read this orientation before answering

The program's internal names are intentionally opaque (`x3`, `x28`, `x167`, etc.), but the external interface is comparatively clear. You should read from the outside inward: usage text, supported flags, config format, parsing functions, validation, flow dispatch, and output writing. Treat the user-facing flags and config keys as the program's API surface.

Concept	Where to look	What to infer
Object specification	`x28`, default init, key application	The runtime object carries names, input payload, width, flow, target, separator, prefix, and suffix.
Fabric	`x31`, add object, execute collection	A config file can hold multiple object specifications executed in sequence.
Output sink	`x3`, file/stdout init, write helpers	The generator is decoupled from the destination; file and stdout share the same write contract.
Flow dispatch	Flow enum and execution branch	The same object schema can run cartesian, literal, repeat, or reverse behavior.

Concept	Where to look	What to infer
Validation boundary	numeric parser, safe power, object validation	Bad widths, empty names, overflows, and impossible combinations should fail before generation.

Part A - Purpose comprehension

A1. In one sentence, state the program's purpose from the viewpoint of a user running it from a shell.

> Answer:

A2. In one sentence, state the program's purpose from the viewpoint of an API or platform engineer.

> Answer:

A3. What problem does the program solve better than a hard-coded permutation generator? Name two differences.

> Answer:

A4. Why might the author preserve opaque `xN` internal identifiers while making runtime object names user-specified?

> Answer:

Part B - External API surface

Fill in the table. Treat every flag or config key as part of the program's API contract.

API input	Accepted examples or aliases	Effect on execution	Validation concern
`object_name`			
`input_value`			
`input_width`			
`flow_type`			
`output_target`			
`separator` / `prefix` / `suffix`			

Part C - Architecture trace

Draw arrows or write a numbered path showing how a value moves from user input to output. Use the function and structure names even if they are opaque.

Example path to complete:

```
CLI flag or config line -> parser -> object specification -> validation -> flow
function -> output sink -> file/stdout
```

C1. Trace:

```
--input-value 01 --input-width 3 --flow-type cartesian --output-target out.txt
```

> Answer:

C2. Trace a config-file object that uses `flow_type=reverse` and `output_target=stdout`.

> Answer:

C3. Explain where the program switches from declarative configuration to imperative execution.

> Answer:

Part D - Code-reading focus questions

D1. Bounded copy: What does the helper that copies strings into fixed buffers prevent, and what usability trade-off does it create?

> Answer:

D2. Numeric parsing: Why does the width parser reject negative values and trailing junk?

> Answer:

D3. Escape decoding: What user-facing feature is enabled by translating `\n`, `\t`, `\r`, and `\\`?

> Answer:

D4. Overflow guard: Why must cartesian generation calculate `strlen(input_value) ^ input_width` safely before iterating?

> Answer:

D5. Output abstraction: How does the sink interface make the generator more testable or extensible?

> Answer:

D6. Multi-object config: What is the engineering significance of supporting more than one object in a config file?

> Answer:

Part E - Multiple choice diagnostic

Choose one answer for each. Do not use the answer key until you have written your choices.

E1. Which statement best describes the external API of this program?

- A. The `xN` function names, because they are the only stable user interface.
- B. The CLI flags, config keys, UI prompts, accepted flow names, and output targets.
- C. The local variable names inside each function.
- D. The compiler optimization level.

Choice: ___
Reason: _____

E2. Why is the output sink design API-oriented?

- A. It hides every error from callers.
- B. It couples generation directly to `fopen`.
- C. It defines a small write contract that different destinations can implement.
- D. It prevents stdout output.

Choice: ___
Reason: _____

E3. What is the main purpose of validating a cartesian object's width and input before generation?

- A. To make output alphabetical.
- B. To prevent impossible, overflowing, or meaningless generation work.
- C. To force every flow to write a file.
- D. To remove the need for tests.

Choice: ___
Reason: _____

E4. Which use case best fits the program as written?

- A. Generating test payloads or fixture files from a declared input alphabet and flow.
- B. Replacing an HTTP server framework.
- C. Encrypting API keys at rest.
- D. Proving memory safety automatically.

Choice: ___
Reason: _____

E5. What does support for config files add beyond one-off CLI flags?

- A. A repeatable declarative artifact that can be versioned, reviewed, and run again.
- B. Guaranteed faster generation for every input.
- C. Elimination of all validation logic.
- D. Removal of stdout support.

Choice: ___
Reason: _____

Part F - API-oriented use-case design

Design three realistic use cases. Each use case must name: user persona, API surface used, flow type, output target, and risk/limit.

Use case	Persona	Interface	Flow	Output	Risk/limit
1					
2					
3					

Part G - Design review and improvement

G1. Identify two strengths of the implementation for API-oriented engineering.

> Answer:

G2. Identify two weaknesses or maintainability risks. At least one must relate to `xN` identifiers, fixed buffer sizes, or error reporting.

> Answer:

G3. Propose one backward-compatible change that improves the API contract. Explain why it does not break existing users.

> Answer:

G4. Propose one non-backward-compatible change that might still be worthwhile in a v3 release.

> Answer:

Part H - Test plan

Write a small test plan. Include at least one unit-level test, one integration test, one failure-path test, and one boundary test.

Test type	Input	Expected result	Why it matters
Unit			
Integration			
Failure path			
Boundary / overflow			
Regression			

Part I - Constructed response

I1. Write a 250-350 word explanation answering this prompt:

> Explain the purpose of `configurator_v2.c` and evaluate how it could be used in API-oriented software engineering.

Your answer must discuss the CLI/config/UI surfaces, the object specification, flow dispatch, output abstraction, validation, and at least two concrete use cases.

Scoring rubric for Part I - 20 points

Criterion	Points	Evidence of mastery
Purpose	4	Accurately describes a user-specifiable generator/fabric rather than a single hard-coded algorithm.
API surface	4	Explains CLI flags, config keys, micro UI, accepted flow names, and outputs as external contract.
Architecture	4	Connects object spec, validation, flow dispatch, and output sink.
Use cases	4	Gives plausible API-engineering uses such as fixture generation, payload combinatorics, config-driven batch generation, docs/demo artifacts, or golden-file testing.
Critical evaluation	4	Mentions risks or improvements: naming opacity, bounded buffers, overflow limits, clearer diagnostics, extensible sink registration, or machine-readable errors.

Answer key and instructor notes

Use this section after completing the worksheet.

Part A - expected themes

- User view: a command-line/configurable tool that writes generated text outputs based on a named object, input payload/alphabet, width, flow type, and output target.
- API/platform view: a small declarative generation engine whose external contract is flags, config keys, supported flows, and output destinations.
- Better than a hard-coded generator: it supports multiple flows, config files, stdout/files, prefixes/suffixes/separators, validation, and multi-object batches.
- Opaque internal names can preserve compatibility with an earlier naming convention while allowing meaningful runtime names for users, though this harms maintainability.

Part B - sample completions

Input	Accepted examples / aliases	Effect	Validation concern
`object_name`	`--object-name`, `-n`, `object`, `name`	Names the runtime object in the spec.	Must fit fixed buffer and not be empty.
`input_value`	`--input-value`, `-i`, `input`, `alphabet`	Payload or alphabet used by the selected flow.	Must fit buffer; cartesian cannot use empty input.
`input_width`	`--input-width`, `-w`, `width`, `length`, `depth`	Depth/count/width for generation.	Reject negatives, junk, zero; cartesian bounded by buffer and overflow guard.
`flow_type`	`cartesian`/`product`/`combinations`, `literal`/`echo`, `repeat`, `reverse`	Selects generation behavior.	Unknown values fail.
`output_target`	`--output-target`, `-o`, `output`, `file`, `stdout`, `-`	Chooses file path or stdout sink.	File open can fail; target must not be empty.
`separator`/`prefix`/`suffix`	`--separator`, `--prefix`, `--suffix`, escapes	Controls record formatting around each generated item.	Escaped text must fit destination buffers.

Part C/D - architecture notes

- CLI/config/UI all populate an `x28` object specification. A collection of objects is stored in `x31`.
- Validation happens before execution; cartesian generation checks buffer-relevant width, non-empty input, and safe power calculation.
- The flow parser maps user strings to enum values; execution dispatch calls cartesian, literal, repeat, or reverse generation.
- The write path goes through a sink object with function pointers for string and character writes, allowing file and stdout behavior through one contract.

- Config-file support makes the program closer to an API automation primitive: configurations can be checked into version control, reviewed, rerun, and used by CI jobs.

Part E - MCQ answers with rationales

E1: B. The external API is the user-facing contract: flags, config keys, UI prompts, accepted names, and targets. The `xN` internals are implementation details.

E2: C. A sink is a small interface that can be implemented by files, stdout, memory buffers, network adapters, or test doubles.

E3: B. Cartesian output can explode combinatorially; validation prevents impossible, overflowing, or meaningless work.

E4: A. The existing behavior is well suited to payload, fixture, and golden-file generation. It is not a server framework, encryption system, or formal verifier.

E5: A. A config file is a repeatable declarative artifact. It improves reviewability, versioning, and automation compared with ad hoc shell commands.

Part F - plausible use cases

- API test fixture generation: use cartesian flow to produce combinations of allowed path/query/header characters for endpoint tests.
- Golden-file generation: use literal, repeat, or reverse flows to create deterministic outputs checked into tests.
- Documentation/demo generation: create sample payload files from config so examples stay repeatable.
- Contract fuzzing seed generation: produce bounded token combinations for validating parsers, provided output size is controlled.
- Batch artifact generation: define multiple objects in one config and run them as a small generation pipeline.

Part G/H - review and testing notes

- Strengths: declarative config, stable CLI surface, sink abstraction, explicit validation, bounded copies, overflow checks, multiple flow modes.
- Weaknesses: opaque internal names, fixed buffer truncation/failure behavior, limited diagnostics, no plugin registry for flows/sinks, no structured machine-readable errors.
- Backward-compatible improvement: add `--dry-run`, `--json-errors`, or a memory sink for tests without removing existing flags.
- Possible v3 breaking change: replace `xN` internal identifiers with descriptive names, reorganize objects into public/private headers, or define a formal config schema.
- Useful tests: parser accepts aliases; parser rejects negative width; cartesian 2 symbols width 3 creates 8 records; stdout target does not close stdout; overflow guard rejects unsafe powers; config with two objects executes both.

Source note

Exercise based on the uploaded advanced curriculum file `curriculum_advanced.pdf` and the Learning Engine instruction-set expectations for comprehension questions, MCQ rationales, constructed response, rubric, and mastery-oriented review.

Exercise: Reading the Enhanced Curriculum C Program for Engineering Comprehension

Purpose

This exercise guides you through a close reading of the **enhanced curriculum** C source code. The goal is not only to understand what the program does, but to evaluate its capabilities as a software-engineering artifact and as an engineering-design solution.

The program is an obfuscated-style C implementation using `xN` identifiers. Treat this as a reverse-engineering and design-comprehension task: infer roles, reconstruct intent, and judge the architecture from evidence in the code.

Learning outcomes

By completing this exercise, you should be able to:

1. Explain the program's observable capability in plain engineering language.
2. Map obfuscated identifiers to meaningful conceptual names.
3. Describe how the program generates fixed-width combinations from an input alphabet.
4. Explain why fixed-width integer types and overflow checks matter.
5. Identify the output abstraction and explain why it improves testability and portability.
6. Evaluate the program against software-engineering qualities: correctness, safety, modularity, maintainability, extensibility, and testability.
7. Propose engineering-design improvements while preserving the program's core behaviour.

Source under study

Use the enhanced curriculum C source code, headed as `configure.md` / `Curriculum`, containing:

- fixed-width integer use via `uint64_t`
- a digit alphabet
- a maximum output width constant
- an output sink structure with function pointers
- a safe exponentiation function
- a permutation/combination generator
- a file-backed output implementation
- a `main` function that generates width-7 digit strings into `x33.txt`

Part 1 — First-pass program comprehension

Read the whole source once without renaming anything.

Answer the following:

1. What file does the program write to?
2. What alphabet does it use?
3. What output width is selected in `main`?
4. How many output records should be produced for that width?
5. What is the shape of each output record?
6. What condition causes `main` to reject the selected width before opening the output file?
7. What does the program return when file opening fails?
8. What does the program return when generation fails after the file has been opened?

Expected reasoning

The answer should connect the constants, alphabet size, selected width, output loop, and newline emission. Do not simply say “it generates permutations”; describe the exact finite language produced by the program.

Part 2 — Identifier de-obfuscation table

Create a table that maps each identifier to its inferred purpose.

Identifier	Inferred name	Evidence from code	Engineering role
`x0`			
`x1`			
`x2`			
`x3`			
`x4`			
`x5`			
`x8`			
`x10`			
`x15`			
`x23`			
`x25`			
`x26`			
`x28`			
`x30`			
`x31`			
`x32`			

Challenge

After completing the table, rewrite the program's architecture in one paragraph using meaningful names only. Example style:

> The program defines a bounded alphabet and a maximum width, computes the number of combinations safely, enumerates each combination by interpreting the loop index in base `alphabet_size`, and writes each generated record through an output-sink abstraction backed by a file.

Part 3 — Capability model

Construct a capability model for the program.

3.1 Functional capabilities

Describe what the program can do.

Minimum points to cover:

- It can generate every fixed-width sequence over a fixed alphabet.
- It can write generated records to a file.
- It can detect arithmetic overflow before computing the total number of records.
- It can stop generation if output fails.
- It can reject invalid requested widths.

3.2 Non-capabilities

Describe what the program cannot currently do.

Consider:

- runtime selection of alphabet
- runtime selection of output width
- runtime selection of output file
- resumable generation
- progress reporting
- streaming to stdout
- configurable separator
- duplicate alphabet detection
- test harness or dependency injection beyond the sink interface

3.3 Engineering implication

For each non-capability, state whether it is:

- a deliberate simplification,
- a safety constraint,
- a maintainability issue,
- a usability limitation,
- or an extensibility opportunity.

Part 4 — Algorithmic comprehension

The core generator uses this pattern:

```
for each row index i:
    temp = i
    for each output position from right to left:
        char_index = temp % alphabet_size
        output[position] = alphabet[char_index]
        temp = temp / alphabet_size
```

Answer:

1. Why does the inner loop fill the output buffer from right to left?
2. What numeral system is being used to convert `i` into a sequence of alphabet indexes?
3. For alphabet `{0,1,2}` and width `2`, list the first six outputs in order.
4. What would change if the inner loop filled left to right without changing the arithmetic?
5. Why is the output buffer not null-terminated before being written?
6. Why is it safe to write exactly `width` bytes from the buffer?
7. Under what condition would this buffer-write approach become unsafe?

Part 5 — Arithmetic safety and overflow design

Study the safe power function.

```
result = 1
repeat exp times:
    if result > UINT64_MAX / base:
        fail
    result *= base
```

Answer:

1. What overflow is this guard preventing?
2. Why is division used before multiplication?
3. What precondition is silently assumed about `base`?
4. Does the current program satisfy that precondition?
5. What would happen if `base == 0`?
6. Should the safe power function itself check for `base == 0`? Defend your answer.
7. What is the largest valid width for alphabet size `10` in `uint64_t` arithmetic?
8. Why does the program still impose `x0 == 10` even though `uint64_t` could represent `10^19`?

Design note

Separate **numeric representability** from **buffer capacity** and **practical output volume**. A value can fit in `uint64_t` while still being impossible or unreasonable to generate in real time.

Part 6 — Output abstraction and design quality

The program defines a structure containing:

- a generic context pointer
- a function pointer for writing a buffer
- a function pointer for writing a character

Answer:

1. What design pattern does this resemble in C?
2. What concrete output implementation is provided?
3. How does this abstraction separate generation logic from file I/O?
4. How could you implement a memory-backed sink for unit tests?
5. How could you implement a counting sink that verifies the number of records without writing a file?
6. What failure path is enabled by returning `bool` from the sink functions?
7. What resource-management responsibility remains outside the abstraction?

Mini-design task

Sketch a `CountingSink` design in pseudocode. It should count:

- bytes written
- newline characters written
- number of records emitted

Do not write full C unless required by your instructor.

Part 7 — File I/O correctness

Inspect the file-backed sink.

Answer:

1. What mode is used to open the file?
2. What happens to any existing file with the same name?
3. How does the buffer-write function check success?
4. How does the character-write function check success?
5. Where is the file closed?
6. Is the file closed if generation fails?
7. Is the file closed if opening the file fails?
8. What risks remain around `fclose` errors?
9. Should `file_sink_close` return a status? Explain the trade-off.

Part 8 — Engineering-design review

Score the program from 1 to 5 in each category and justify each score with code evidence.

Category	Score	Evidence	Improvement
Correctness			
Arithmetic safety			
Memory safety			
I/O error handling			
Modularity			
Testability			
Maintainability			
Configurability			
Scalability			
Usability			

Guidance

A high score requires evidence, not optimism. A low score should identify a concrete engineering improvement.

Part 9 — Software-engineering capability statement

Write a concise capability statement in this format:

> This program is capable of [capability] under [constraints]. It achieves this through [architectural mechanisms]. Its strongest engineering features are [features]. Its primary limitations are [limitations]. It would be suitable for [use cases] but unsuitable for [use cases] unless [changes].

Your statement should mention:

- finite exhaustive generation
- bounded width
- safe arithmetic guard
- sink abstraction
- file output
- lack of runtime configurability
- impracticality of large output spaces

Part 10 — Engineering-design extension proposal

Propose one extension that improves the program without destroying its safety properties.

Choose one:

1. CLI-configurable width and output path

2. runtime-configurable alphabet
3. stdout sink
4. memory sink for testing
5. progress reporting
6. resumable generation
7. output-size preflight warning
8. duplicate-character detection in the alphabet

For your chosen extension, provide:

- user story
- new inputs
- validation rules
- affected functions
- new failure modes
- test cases
- safety argument

Part 11 — Test design

Design tests for the program.

Required test categories

1. **Unit tests for safe power**
    - small valid powers
    - exponent zero
    - boundary before overflow
    - overflow detection
    - base zero behaviour
2. **Generator tests using a fake sink**
    - width 1 over a small alphabet
    - width 2 over a small alphabet
    - sink write failure on record N
    - sink newline failure
    - verify generation stops after failure
3. **File sink tests**
    - invalid path
    - successful file creation
    - expected file size for a small configuration
    - close behaviour
4. **Main-level behaviour**
    - valid configured width

- zero width rejection
- width exceeding maximum rejection

Deliverable

Write a test matrix:

Test ID	Unit under test	Input/setup	Expected result	Engineering property verified
T1				
T2				
T3				

Part 12 — Risk analysis

Complete a risk table.

Risk	Cause	Impact	Existing mitigation	Proposed improvement
Very large output file				
Arithmetic overflow				
Buffer overrun				
File-open failure				
Mid-generation write failure				
Ambiguous obfuscated identifiers				
Inflexible hard-coded settings				
Silent close failure				

Part 13 — Refactoring task

Refactor the naming only. Do not change behaviour.

Produce a mapping and then rewrite the following as meaningful C names:

- constants
- alphabet array
- sink structure
- safe power function
- generator function
- file sink functions
- local variables in the generator
- `main` constants and sink variable

Constraints

- Preserve the same generated output.
- Preserve the same return-code behaviour.
- Preserve the same file target.
- Preserve the same arithmetic guard.
- Preserve the same maximum width.

Part 14 — Written reflection

Answer in 300–500 words:

> What does this program reveal about the relationship between low-level C programming, software-engineering discipline, and engineering design?

Your answer should discuss:

- explicit bounds
- abstraction through function pointers
- error propagation
- resource lifetime
- algorithmic scaling
- the difference between a working program and an engineered program

---

Part 15 — Mastery check

Answer without looking back at your notes.

1. What is the program's core algorithm?
2. What prevents integer overflow?
3. What prevents buffer overflow?
4. What abstraction separates generation from output?
5. What hard-coded choices limit the program's use?
6. What is the first extension you would implement and why?
7. What is the strongest evidence that the program was written with safety-critical design principles in mind?
8. What is the strongest evidence that the program is still only a small curriculum example rather than a production tool?

Optional advanced challenge

Compare this enhanced curriculum program with a more advanced configurable version.

Focus on how the design changes when these capabilities are added:

- CLI flags
- config-file parsing
- micro UI
- multiple flow types

- stdout output
- prefix, suffix, and separator handling
- multiple object specifications

Write a short comparison:

Design aspect	Enhanced curriculum program	Advanced configurable program	Engineering consequence
Input configuration			
Flow selection			
Output targeting			
Validation burden			
Test surface			
Maintainability			
User capability			

Submission checklist

Submit:

- completed identifier mapping table
- capability and non-capability model
- algorithm explanation
- safety analysis
- design-quality score table
- one extension proposal
- test matrix
- risk table
- 300–500 word reflection

Evaluation rubric

Criterion	Excellent	Satisfactory	Needs work
Program comprehension	Accurately explains exact output language and control flow	Explains general purpose but misses details	Vague or incorrect description
Identifier mapping	Most identifiers mapped with evidence	Some mappings correct	Mostly guessed
Safety reasoning	Clearly separates arithmetic, buffer, and I/O safety	Mentions safety but lacks detail	Treats all safety issues as the same

Criterion	Excellent	Satisfactory	Needs work
Architecture analysis	Explains sink abstraction and modularity	Identifies functions but not architecture	Only describes syntax
Engineering judgement	Balanced trade-off analysis	Some useful recommendations	Unsupported opinions
Test design	Covers unit, integration, failure, and boundary cases	Covers happy paths and some boundaries	Minimal or no failure testing
Extension proposal	Preserves safety and defines validation/failure modes	Plausible but incomplete	Feature idea without design
Reflection	Connects C details to engineering design principles	General reflection	Superficial summary

Instructor notes / answer anchors

Use these only after attempting the exercise.

- The program enumerates fixed-width strings over the digit alphabet `0`–`9`.
- With width `7`, it attempts to generate `10^7` records.
- Each record is seven characters followed by a newline.
- The output target is `x33.txt`.
- The generator interprets the loop counter as a base-10 number over the alphabet indexes.
- The sink abstraction is a C form of dependency inversion: the generator does not know whether output goes to a file, memory, stdout, or a test double.
- The overflow guard checks multiplication before performing it.
- The maximum width protects the fixed-size stack buffer.
- The program is more engineered than a quick script because it uses bounded buffers, explicit failure returns, fixed-width integers, and output abstraction.
- The program is not a production tool because key parameters are hard-coded, close errors are ignored, huge output is not preflighted, and there is no built-in test harness.

Boolean Truth Table Exercise — Selecting a Generative Hardware Design Toolchain

Purpose. Decide which of three curriculum-derived programs, or which combined toolchain, best serves as a manual/semi-automated generative program for **Hardware Design Engineering** using **Python + KiCad**, three-dimensional block design, superposition, dimension matching, and collision theory.

> This is an exercise, not a certified manufacturing workflow. The demo is deliberately educational: it generates abstract block envelopes, interface points, fit checks, and optional KiCad placement scaffolding. Real manufacture requires datasheets, tolerances, materials, insulation ratings, thermal analysis, electrical safety review, and DFM/DFA sign-off.

---

1. The Three Candidate Programs

Symbol	Program	Interpreted role in this exercise	Main design pattern extracted
A	Program 1: baseline safety-critical permutation generator	Clean, auditable generator core	`Configuration → OutputSink → safe bounded generation → FileSink`
B	Program 2: enhanced compact fixed-width generator	Minimal parametric enumerator	Fixed-width decimal/token generation with bounded output
C	Program 3: advanced `configurator_v2` fabric	User-specifiable generative fabric	Config/CLI/UI-defined objects, flows, separators, prefix/suffix, targets

---

2. Boolean Feature Predicates

Evaluate each program or program-combination against these predicates.

Predicate	Meaning	Hardware-design interpretation
G	General generative breadth	Can generate many alternatives, not just one fixed object
Cnf	Configurability	User can specify objects, dimensions, flows, outputs
S	Safety/verification posture	Uses bounded arithmetic, validation, preconditions, fail-fast behaviour
I	Interface abstraction	Has a separable sink/output or tool boundary
K	KiCad/Python bridgeability	Can be mapped naturally into Python objects and KiCad footprints/placement
D	Dimension matching	Can represent width/height/depth and connector/interface matching
X	Collision checking	Can test 3D/2D overlap before assembly

Predicate	Meaning	Hardware-design interpretation
M	Manufacture/assembly metadata	Can preserve naming, prefix/suffix, target, process notes, or export artifacts

A candidate is considered **hardware-toolchain-ready** when:

```
READY = G ∧ Cnf ∧ S ∧ I ∧ K ∧ D ∧ X ∧ M
```

A candidate is considered **teaching-ready but not production-ready** when:

```
TEACHING = (S ∧ I) ∨ (G ∧ D ∧ X)
```

---

3. Boolean Truth Table

Use `1 = present/satisfied`, `0 = absent/not sufficiently satisfied`.

Row	A	B	C	Candidate toolchain	G	Cnf	S	I	K	D	X	M	READY	TEACHING	Decision
1	1	0	0	**A only**	1	0	1	1	1	0	0	0	0	1	Good safety kernel, weak hardware configurability
2	0	1	0	**B only**	1	0	1	1	1	0	0	0	0	1	Minimal generator, useful for numeric series and IDs
3	0	0	1	**C only**	1	1	1	1	1	1	0	1	0	1	Best single program, but needs explicit collision engine
4	1	1	0	**A+B**	1	0	1	1	1	0	0	0	0	1	Strong bounded enumerator pair; not enough object metadata
5	1	0	1	**A+C**	1	1	1	1	1	1	0	1	0	1	Strongest two-program base; add

Row	A	B	C	Candidate toolchain	G	Cnf	S	I	K	D	X	M	READY	TEACHING	Decision
															collision module
6	0	1	1	**B+C**	1	1	1	1	1	1	0	1	0	1	Good compact + configurable fabric; add safety audit from A
7	1	1	1	**A+B+C hybrid**	1	1	1	1	1	1	1	1	1	1	**Best choice: hybrid hardware design generator**

Answer Key

The best answer is:

```
A ∧ B ∧ C = READY
```

The **hybrid toolchain** is best because it combines:

1. **A:** auditable safety-critical generator structure, overflow guards, and output abstraction.
2. **B:** compact deterministic fixed-width enumeration, useful for component codes, pin labels, footprints, and part variants.
3. **C:** configurable object/flow fabric, useful for user-named hardware objects, config files, CLI/UI invocation, and flexible outputs.
4. **Python + KiCad layer:** adds the missing hardware-specific geometry, interface, 3D envelope, collision, and board-placement semantics.

---

4. Hardware Design Pattern Philosophy

The exercise treats each electrical component as a **design pattern instance**.

```
Component Pattern =
    identity
  + functional role
  + 3D envelope
  + interface points
  + material/process metadata
  + electrical constraints
  + assembly constraints
  + collision/dimension checks
  + KiCad footprint/placement output
```

Component set to build

```
wire
cable
connector
battery cell
supercapacitor
switch
relay
resistor
capacitor
inductor
transformer
fuse
circuit breaker
```

Four engineering lenses

Lens	Question	Example
Superposition	What happens when candidate layouts are overlaid in the same coordinate frame?	Overlay connector, fuse, relay, and battery keep-out zones.
Dimension matching	Do interfaces physically and electrically match?	Cable diameter matches connector bore; pin pitch matches footprint pads.
Collision theory	Do occupied volumes intersect?	Relay case must not intersect transformer volume or battery keep-out.
DFM/DFA	Can it be manufactured and assembled?	Sufficient clearance, labeled orientation, process notes, access space.

---

5. Interactive Demo — Python Geometry + Optional KiCad Export

What the demo does

The demo builds a tiny abstract hardware assembly:

- Models each required component as a 3D axis-aligned block.
- Adds interface points such as terminals, pins, or cable endpoints.
- Checks **dimension matching** between paired interfaces.
- Checks **collision** with AABB overlap.
- Produces a simple placement manifest.
- Optionally creates a KiCad PCB scaffold if run inside an environment where `pcbnew` is available.

Install/use notes

- Geometry-only demo runs with standard Python 3.
- KiCad export section is optional and version-sensitive.
- For modern KiCad plugin work, prefer KiCad's IPC API / `kicad-python` where available.
- Legacy `pcbnew` scripting examples are included as a fallback pattern for learning.

---

6. Demo Code

Save as:

```
hardware_block_demo.py
```

```
from __future__ import annotations

from dataclasses import dataclass, field
from itertools import combinations
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Tuple
import json
import math

Vec3 = Tuple[float, float, float]


@dataclass(frozen=True)
class Interface:
    """A physical/electrical connection point on a component block."""
    name: str
    kind: str                 # e.g. wire_end, pin, lug, terminal, pad
    position: Vec3            # local component coordinates in mm
    nominal_diameter: float   # mm
    voltage_class: str = "ELV"
    current_class: str = "signal"


@dataclass
class BlockComponent:
    """3D block approximation of a hardware component."""
    name: str
    category: str
    origin: Vec3              # world-space lower-left-bottom corner in mm
    size: Vec3                # width, depth, height in mm
    interfaces: List[Interface] = field(default_factory=list)
    material: str = "unspecified"
    process: str = "manual assembly"
    notes: str = ""

    @property
    def max_corner(self) -> Vec3:
        return tuple(self.origin[i] + self.size[i] for i in range(3))  # type: ignore

    def world_interface(self, iface: Interface) -> Vec3:
        return tuple(self.origin[i] + iface.position[i] for i in range(3))  # type:
ignore


def aabb_collides(a: BlockComponent, b: BlockComponent, clearance: float = 0.0) ->
bool:
    """Collision theory: axis-aligned bounding-box overlap with optional clearance."""
```

```
    amin, amax = a.origin, a.max_corner
    bmin, bmax = b.origin, b.max_corner

    return all(
        (amin[i] - clearance) < bmax[i] and
        (amax[i] + clearance) > bmin[i]
        for i in range(3)
    )


def distance(p: Vec3, q: Vec3) -> float:
    return math.sqrt(sum((p[i] - q[i]) ** 2 for i in range(3)))


def dimension_match(a: Interface, b: Interface, tolerance: float = 0.25) -> bool:
    """Dimension matching: simple round-interface compatibility."""
    return abs(a.nominal_diameter - b.nominal_diameter) <= tolerance


def make_component_library() -> Dict[str, BlockComponent]:
    """Abstract 3D block library for all requested components."""
    return {
        "wire": BlockComponent(
            "wire", "conductor", (0, 0, 0), (30, 1, 1),
            [Interface("end_A", "wire_end", (0, 0.5, 0.5), 1.0),
             Interface("end_B", "wire_end", (30, 0.5, 0.5), 1.0)],
            material="copper + insulation",
            process="cut, strip, crimp/solder"
        ),
        "cable": BlockComponent(
            "cable", "multi_conductor", (0, 4, 0), (40, 3, 3),
            [Interface("end_A", "cable_end", (0, 1.5, 1.5), 3.0),
             Interface("end_B", "cable_end", (40, 1.5, 1.5), 3.0)],
            material="copper bundle + jacket",
            process="cut, strip, strain-relief"
        ),
        "connector": BlockComponent(
            "connector", "interconnect", (42, 4, 0), (8, 6, 5),
            [Interface("bore", "socket", (0, 3, 2.5), 3.1),
             Interface("pin_1", "pad", (7, 2, 0), 1.0),
             Interface("pin_2", "pad", (7, 4, 0), 1.0)],
            material="polymer housing + plated contacts",
            process="insert, solder, inspect"
        ),
        "battery_cell": BlockComponent(
            "battery_cell", "energy_storage", (0, 14, 0), (18, 65, 18),
            [Interface("positive", "terminal", (18, 32.5, 9), 5.0, "battery"),
             Interface("negative", "terminal", (0, 32.5, 9), 5.0, "battery")],
            material="cell chemistry dependent",
            process="holder or welded tabs; observe polarity"
        ),
        "supercapacitor": BlockComponent(
            "supercapacitor", "energy_storage", (24, 14, 0), (10, 20, 18),
            [Interface("positive", "lead", (5, 3, 0), 0.8),
             Interface("negative", "lead", (5, 17, 0), 0.8)],
            material="aluminium can",
```

```
            process="through-hole insert; polarity check"
        ),
        "switch": BlockComponent(
            "switch", "control", (55, 0, 0), (12, 6, 5),
            [Interface("common", "pad", (2, 3, 0), 1.0),
             Interface("throw", "pad", (10, 3, 0), 1.0)],
            material="plastic actuator + metal contacts",
            process="panel/PCB mount"
        ),
        "relay": BlockComponent(
            "relay", "electromechanical_control", (72, 0, 0), (20, 15, 12),
            [Interface("coil_A", "pad", (3, 3, 0), 1.0),
             Interface("coil_B", "pad", (3, 12, 0), 1.0),
             Interface("contact_COM", "pad", (17, 3, 0), 1.2),
             Interface("contact_NO", "pad", (17, 12, 0), 1.2)],
            material="sealed relay package",
            process="PCB insert; creepage/clearance review"
        ),
        "resistor": BlockComponent(
            "resistor", "passive", (42, 16, 0), (8, 3, 3),
            [Interface("lead_A", "lead", (0, 1.5, 1.5), 0.6),
             Interface("lead_B", "lead", (8, 1.5, 1.5), 0.6)],
            material="film or wirewound",
            process="place, solder, trim"
        ),
        "capacitor": BlockComponent(
            "capacitor", "passive", (42, 24, 0), (8, 5, 8),
            [Interface("lead_A", "lead", (3, 2.5, 0), 0.6),
             Interface("lead_B", "lead", (5, 2.5, 0), 0.6)],
            material="ceramic/electrolytic/film",
            process="place; polarity if required"
        ),
        "inductor": BlockComponent(
            "inductor", "magnetic", (55, 24, 0), (12, 10, 8),
            [Interface("lead_A", "lead", (0, 5, 0), 0.8),
             Interface("lead_B", "lead", (12, 5, 0), 0.8)],
            material="winding + core",
            process="place; magnetic clearance"
        ),
        "transformer": BlockComponent(
            "transformer", "magnetic", (72, 24, 0), (24, 20, 18),
            [Interface("primary_A", "lug", (2, 3, 0), 1.2),
             Interface("primary_B", "lug", (2, 17, 0), 1.2),
             Interface("secondary_A", "lug", (22, 3, 0), 1.2),
             Interface("secondary_B", "lug", (22, 17, 0), 1.2)],
            material="copper winding + laminated/ferrite core",
            process="mount; isolation and thermal review"
        ),
        "fuse": BlockComponent(
            "fuse", "protection", (42, 38, 0), (18, 5, 5),
            [Interface("clip_A", "clip", (0, 2.5, 2.5), 2.0),
             Interface("clip_B", "clip", (18, 2.5, 2.5), 2.0)],
            material="fuse body + clips",
            process="clip holder; replaceability access"
        ),
        "circuit_breaker": BlockComponent(
```

```
            "circuit_breaker", "protection", (64, 38, 0), (28, 18, 20),
            [Interface("line", "terminal", (0, 6, 10), 3.5),
             Interface("load", "terminal", (28, 6, 10), 3.5)],
            material="breaker housing + contacts",
            process="panel mount; torque terminals"
        ),
    }


def superpose_variants(base: Iterable[BlockComponent], dx: float = 0.0, dy: float =
0.0) -> List[BlockComponent]:
    """
    Superposition: overlay or offset candidate assemblies.
    dx=dy=0 means true overlay; nonzero offsets create candidate alternatives.
    """
    out = []
    for item in base:
        moved = BlockComponent(
            name=f"{item.name}_variant",
            category=item.category,
            origin=(item.origin[0] + dx, item.origin[1] + dy, item.origin[2]),
            size=item.size,
            interfaces=item.interfaces,
            material=item.material,
            process=item.process,
            notes=f"superposed offset dx={dx}, dy={dy}"
        )
        out.append(moved)
    return out


def report_collisions(parts: List[BlockComponent], clearance: float = 0.5) ->
List[Tuple[str, str]]:
    hits = []
    for a, b in combinations(parts, 2):
        if aabb_collides(a, b, clearance=clearance):
            hits.append((a.name, b.name))
    return hits


def report_interface_matches(parts: Dict[str, BlockComponent]) -> List[Dict[str,
object]]:
    """Small set of intentional interface fit checks."""
    checks = [
        ("cable", "end_B", "connector", "bore"),
        ("wire", "end_B", "switch", "common"),
        ("fuse", "clip_B", "circuit_breaker", "line"),
        ("supercapacitor", "positive", "capacitor", "lead_A"),
    ]

    rows = []
    for comp_a, iface_a, comp_b, iface_b in checks:
        a = parts[comp_a]
        b = parts[comp_b]
        ia = next(i for i in a.interfaces if i.name == iface_a)
        ib = next(i for i in b.interfaces if i.name == iface_b)
```

```
            rows.append({
                "a": f"{comp_a}.{iface_a}",
                "b": f"{comp_b}.{iface_b}",
                "diameter_a_mm": ia.nominal_diameter,
                "diameter_b_mm": ib.nominal_diameter,
                "dimension_match": dimension_match(ia, ib),
                "world_distance_mm": round(distance(a.world_interface(ia),
b.world_interface(ib)), 2),
            })
    return rows


def export_manifest(parts: Dict[str, BlockComponent], path: Path) -> None:
    data = []
    for part in parts.values():
        data.append({
            "name": part.name,
            "category": part.category,
            "origin_mm": part.origin,
            "size_mm": part.size,
            "interfaces": [
                {
                    "name": i.name,
                    "kind": i.kind,
                    "local_position_mm": i.position,
                    "nominal_diameter_mm": i.nominal_diameter,
                    "voltage_class": i.voltage_class,
                    "current_class": i.current_class,
                }
                for i in part.interfaces
            ],
            "material": part.material,
            "process": part.process,
            "notes": part.notes,
        })
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def optional_kicad_export(parts: Dict[str, BlockComponent], out_path: Path) -> str:
    """
    Optional KiCad scaffold.

    This deliberately uses a conservative legacy pcbnew pattern if available:
    - Create/load a board
    - Add simple text labels as placement proxies
    - Save the board

    In KiCad 9+, modern plugin development should prefer the IPC API / kicad-python
    where possible. This fallback is only a teaching scaffold.
    """
    try:
        import pcbnew  # type: ignore
    except Exception as exc:
        return f"KiCad pcbnew not available; skipped board export: {exc}"

    board = pcbnew.BOARD()
```

```
    for idx, part in enumerate(parts.values()):
        # PCB_TEXT exists in recent pcbnew versions; older versions may differ.
        text = pcbnew.PCB_TEXT(board)
        text.SetText(part.name)
        text.SetPosition(pcbnew.VECTOR2I(int(part.origin[0] * 1_000_000),
int(part.origin[1] * 1_000_000)))
        text.SetLayer(pcbnew.F_SilkS)
        board.Add(text)

    board.Save(str(out_path))
    return f"Saved KiCad scaffold to {out_path}"


def main() -> None:
    parts = make_component_library()

    print("\n=== Hardware Block Manifest ===")
    for p in parts.values():
        print(f"{p.name:18s} origin={p.origin} size={p.size} process={p.process}")

    print("\n=== Dimension Matching Checks ===")
    for row in report_interface_matches(parts):
        print(row)

    print("\n=== Collision Checks with 0.5 mm clearance ===")
    collisions = report_collisions(list(parts.values()), clearance=0.5)
    if not collisions:
        print("No collisions detected.")
    else:
        for a, b in collisions:
            print(f"COLLISION/CLEARANCE FAIL: {a} <-> {b}")

    print("\n=== Superposition Test ===")
    overlay = superpose_variants([parts["relay"], parts["transformer"]], dx=0, dy=0)
    overlay_hits = report_collisions([parts["relay"], parts["transformer"], *overlay],
clearance=0.0)
    for hit in overlay_hits:
        print(f"Superposed overlap: {hit}")

    manifest = Path("hardware_blocks_manifest.json")
    export_manifest(parts, manifest)
    print(f"\nWrote {manifest}")

    print(optional_kicad_export(parts, Path("hardware_blocks.kicad_pcb")))


if __name__ == "__main__":
    main()
```

---

7. Expected Interactive Output Pattern

When you run:

```
python hardware_block_demo.py
```

you should see sections like:

```
=== Hardware Block Manifest ===
wire               origin=(0, 0, 0) size=(30, 1, 1) process=cut, strip, crimp/solder
cable              origin=(0, 4, 0) size=(40, 3, 3) process=cut, strip, strain-relief
...

=== Dimension Matching Checks ===
{'a': 'cable.end_B', 'b': 'connector.bore', 'diameter_a_mm': 3.0, 'diameter_b_mm':
3.1, 'dimension_match': True, ...}
{'a': 'supercapacitor.positive', 'b': 'capacitor.lead_A', 'diameter_a_mm': 0.8,
'diameter_b_mm': 0.6, 'dimension_match': True, ...}

=== Collision Checks with 0.5 mm clearance ===
No collisions detected.
```

If KiCad's Python module is available, it also attempts to write:

```
hardware_blocks.kicad_pcb
```

Otherwise it still writes:

```
hardware_blocks_manifest.json
```

---

8. Design-Manufacture-Assembly Mapping

Component	Design variables	Manufacture variables	Assembly variables	Demo abstraction
Wire	length, gauge, insulation, endpoint type	cut length, strip length, conductor material	bend radius, termination	3D line/block + two wire-end interfaces
Cable	conductor count, jacket diameter, shield	cut, strip, jacket removal	strain relief, connector fit	larger block + cable-end interface
Connector	pitch, pin count, bore/socket size	plated contacts, housing	mating direction, keying	block + bore + pads
Battery cell	chemistry, voltage, capacity, terminals	cell procurement, holder/weld tabs	polarity, spacing, thermal access	energy block + positive/negative terminals

Component	Design variables	Manufacture variables	Assembly variables	Demo abstraction
Supercapacitor	capacitance, ESR, voltage rating	can package, lead forming	polarity, clearance	cylindrical-as-block + leads
Switch	poles/throws, current rating	contact material, actuator	panel or PCB mounting	control block + terminals
Relay	coil voltage, contact rating	sealed package	coil/contact separation	block + coil/contact pads
Resistor	value, tolerance, power	film/wirewound, lead forming	placement, heat spacing	passive block + leads
Capacitor	value, dielectric, polarity	package, forming	polarity, spacing	passive block + leads
Inductor	inductance, current, saturation	winding/core	magnetic clearance	magnetic block + leads
Transformer	turns ratio, isolation, VA	winding, core, impregnation	creepage, thermal, mounting	larger magnetic block + lugs
Fuse	rating, package, holder	fuse/clip selection	replacement access	protection block + clips
Circuit breaker	trip curve, rating, terminals	DIN/panel package	terminal torque, access	protection block + line/load terminals

9. Student Tasks

Task A — Complete the Boolean decision

1. Explain why **C only** almost reaches readiness but fails `X`.
2. Explain why **A+C** is a better two-program pairing than **B+C** for a safety-critical engineering workflow.
3. Explain why the hybrid **A+B+C** becomes ready only after adding the Python/KiCad geometry layer.

Task B — Modify the demo

Change the relay origin to:

```
"relay": BlockComponent(... origin=(72, 24, 0), ...)
```

Then run the demo again.

Question:

```
Which component does it collide with, and what design change would resolve the
collision?
```

Expected reasoning:

```
The relay is likely to collide with the transformer envelope.
Resolve by offsetting one block, changing package size, rotating placement, or
introducing a keep-out zone.
```

Task C — Add a new design rule

Add a rule:

```
energy_storage components must be at least 5 mm away from magnetic components
```

Implement this as a function:

```
def category_clearance_rule(parts, category_a, category_b, required_clearance_mm):
    ...
```

Task D — KiCad extension challenge

Replace the simple text-label export with real footprints or footprint proxies:

1. Create rectangular silkscreen outlines for each block.
2. Add pad proxies for each interface.
3. Store `material`, `process`, and `notes` as text fields or external JSON metadata.
4. Run KiCad DRC manually after import.

---

10. Final Recommendation

```
Best generative hardware design toolchain:
    A+B+C hybrid + Python geometry kernel + KiCad export/placement layer
```

Boolean form:

```
READY = A ∧ B ∧ C ∧ PythonGeometry ∧ KiCadBridge
```

Engineering form:

```
safe generator
+ compact deterministic enumerator
+ configurable fabric
+ 3D block model
+ dimension-matching rules
+ collision/clearance rules
```

```
+ KiCad placement/export
= manual/semi-automated hardware design engineering toolchain
```

The important design insight is that the C programs supply **generative computation patterns**, while Python and KiCad supply the **hardware-engineering embodiment**: geometry, interfaces, footprints, placement, documentation, and review.